

Оптимизации spark

Введение

В данной работе исследуются типичные антипаттерны производительности Apache Spark и демонстрируются способы их устранения. Для бенчмаркинга использовался pytest-benchmark с синтетическим AdTech датасетом (20 млн записей impressions, 200 geo-регионов).

Окружение

Параметр	Значение
CPU	Apple M2 Pro (12 cores)
RAM	16 GB
Python	3.13.7
Spark	local[*] mode

Датасет

Для бенчмаркинга был сгенерирован синтетический AdTech датасет, имитирующий RTB лог. Генератор: `src/bin/generate.rs`

Основная таблица: `impressions.parquet`

Поле	Тип	Описание
<code>transaction_id</code>	String	Уникальный ID транзакции (<code>tx_0</code> , <code>tx_1</code> , ...)
<code>publisher_id</code>	String	ID публишера (владельца рекламной площадки)
<code>geo_id</code>	Int32	ID географического региона (0-199)
<code>bid_price</code>	Float64	Цена ставки в USD (0.00 - 10.00, 5% нулевых)
<code>timestamp</code>	Int64	Unix timestamp
<code>user_agent</code>	String	User-Agent браузера (8 вариантов: iPhone, Android, Windows, Mac, iPad, Linux, Firefox)
<code>ip_address</code>	String	IP-адрес (10% internal 192.168.x.x, 5% localhost 10.0.x.x, 85% external)
<code>bid_request</code>	String	JSON с параметрами запроса

Размер: 20 000 000 строк, ~800 MB (Parquet со Snappy сжатием)

Структура `bid_request` JSON

```
{
  "ad_size": "300x250",
  "floor_price": 1.23,
  "domain": "news.com",
  "viewability": 75,
  "gdpr_consent": true
}
```

Справочник: **geo_dict.parquet**

Поле	Тип	Описание
geo_id	Int32	ID региона (0-199)
country_code	String	Код страны (RU, US, DE, FR, GB, CN_5...CN_199)

Бенчмарк 1: Shuffle Join vs Broadcast Join

```
def test_shuffle_join(benchmark, dataframes):
    df_imp, df_geo = dataframes

    def run_shuffle_join():
        joined = df_imp.join(df_geo, "geo_id")
        joined.write.format("noop").mode("overwrite").save()

    benchmark(run_shuffle_join)

def test_broadcast_join(benchmark, dataframes):
    df_imp, df_geo = dataframes

    def run_broadcast_join():
        joined = df_imp.join(broadcast(df_geo), "geo_id")
        joined.write.format("noop").mode("overwrite").save()

    benchmark(run_broadcast_join)
```

Результаты (10M строк, 200 shuffle partitions)

Метод	Mean (s)	Median (s)	StdDev	Speedup
Shuffle Join	21.20	20.82	4.00	1.0x (baseline)
Broadcast Join	9.56	9.40	0.56	2.2x

Broadcast join показал **ускорение в 2.2 раза** по сравнению с shuffle join. При broadcast join маленькая таблица **geo_dict** (200 строк) рассылается на все ехесутор'ы, и соединение выполняется локально без shuffle.

Бенчмарк 2: Избыточные партии после фильтрации

Проблема

После фильтрации данных количество партий остаётся прежним, но данных в них становится значительно меньше. Spark запускает множество микроскопических задач, где оверхед планировщика превышает полезную работу.

```
def test_many_small_partitions(benchmark, data):
    def run_fragmented():
        filtered = data.filter(~F.col("publisher_id").isin(
            ["publisher_whale", "publisher_medium_fish"]))

        result = filtered.groupBy("geo_id").agg(
            F.sum("bid_price").alias("total_bid"),
            F.count("*").alias("count")
        )
        result.write.format("noop").mode("overwrite").save()

    benchmark(run_fragmented)

def test_coalesced_partitions(benchmark, data):
    def run_coalesced():
        filtered = data.filter(~F.col("publisher_id").isin(
            ["publisher_whale", "publisher_medium_fish"]))

        coalesced = filtered.coalesce(12)

        result = coalesced.groupBy("geo_id").agg(
            F.sum("bid_price").alias("total_bid"),
            F.count("*").alias("count")
        )
        result.write.format("noop").mode("overwrite").save()

    benchmark(run_coalesced)

def test_repartition_vs_coalesce(benchmark, data):
    def run_repartition():
        filtered = data.filter(~F.col("publisher_id").isin(
            ["publisher_whale", "publisher_medium_fish"]))

        repartitioned = filtered.repartition(12)

        result = repartitioned.groupBy("geo_id").agg(
            F.sum("bid_price").alias("total_bid"),
            F.count("*").alias("count")
        )
        result.write.format("noop").mode("overwrite").save()

    benchmark(run_repartition)
```

Результаты

Метод	Mean (s)	Median (s)	StdDev	Speedup vs baseline
200 мелких партиций	4.28	4.11	0.59	1.0x (baseline)
Coalesce до 12	0.63	0.62	0.09	6.8x
Repartition до 12	2.80	2.55	0.40	1.5x

Анализ

Coalesce показал **ускорение в 6.8 раз!** Это один из самых значимых результатов:

- **200 партиций:** Spark запускает 200 задач, каждая обрабатывает ~15K строк. Оверхед планировщика доминирует.
- **Coalesce (12 партиций):** 12 задач по ~250K строк каждая. Эффективное использование CPU. **Без shuffle!**
- **Repartition (12 партиций):** Тоже 12 задач, но с полным shuffle — в **4.4x медленнее** coalesce.

Бенчмарк 3: Python UDF vs Native Functions

Проблема

Python UDF требуют сериализации данных из JVM в Python и обратно для каждой строки. Это создаёт огромный оверхед по сравнению с нативными функциями Spark, которые выполняются полностью в JVM.

```
def categorize_bid(price):
    if price is None: return "unknown"
    elif price == 0: return "zero"
    elif price < 2.0: return "low"
    elif price < 5.0: return "medium"
    else: return "high"

categorize_udf = F.udf(categorize_bid, StringType())

def test_python_udf(benchmark, data):
    def run_udf():
        result = data.withColumn("bid_category",
                                categorize_udf(F.col("bid_price")))
        result.write.format("noop").mode("overwrite").save()
    benchmark(run_udf)

def test_native_functions(benchmark, data):
    def run_native():
        result = data.withColumn(
            "bid_category",
            F.when(F.col("bid_price").isNull(), "unknown")
              .when(F.col("bid_price") == 0, "zero")
              .when(F.col("bid_price") < 2.0, "low")
```

```
        .when(F.col("bid_price") < 5.0, "medium")
        .otherwise("high")
    )
    result.write.format("noop").mode("overwrite").save()
benchmark(run_native)
```

Результаты (500K строк)

Метод	Mean (s)	Median (s)	StdDev	Speedup
Python UDF	0.373	0.349	0.057	1.0x (baseline)
Pandas UDF	0.277	0.272	0.012	1.3x
Native Functions	0.166	0.163	0.007	2.3x

Анализ

Native функции показали **ускорение в 2.3 раза** по сравнению с Python UDF:

- **Python UDF**: Каждая строка сериализуется JVM → Python → JVM
- **Pandas UDF**: Векторизованная обработка батчами через Arrow (компромисс)
- **Native**: Полностью в JVM, без сериализации

Бенчмарк 4: Комплексная обработка — UDF vs Native

Проблема

Реальные ETL-пайплайны часто содержат несколько трансформаций: парсинг User-Agent, валидация IP, извлечение данных из JSON. Каждый Python UDF добавляет оверхед сериализации.

```
def test_python_udf(benchmark, data):
    device_udf = F.udf(extract_device, StringType()) # Парсинг UA
    internal_udf = F.udf(is_internal_ip, BooleanType()) # Проверка IP
    floor_udf = F.udf(extract_floor_price, DoubleType()) # Парсинг JSON

    def run():
        result = data \
            .withColumn("device_type", device_udf(F.col("user_agent"))) \
            .withColumn("is_internal", internal_udf(F.col("ip_address"))) \
            .withColumn("floor_price", floor_udf(F.col("bid_request")))
        result.write.format("noop").mode("overwrite").save()
    benchmark(run)

def test_native_functions(benchmark, data):
    def run():
        result = data \
            .withColumn("device_type",
                F.when(F.col("user_agent").isNull(), "unknown")
```

```
        .when(F.col("user_agent").contains("iPhone") |
              F.col("user_agent").contains("iPad"), "ios")
        .when(F.col("user_agent").contains("Android"), "android")
        .otherwise("other")) \
    .withColumn("is_internal",
               F.col("ip_address").startswith("192.168.") |
               F.col("ip_address").startswith("10. ")) \
    .withColumn("floor_price",
               F.get_json_object(F.col("bid_request"), "$.floor_price")
               .cast(DoubleType()))
    result.write.format("noop").mode("overwrite").save()
benchmark(run)
```

Результаты (500K строк)

Метод	Mean (s)	Median (s)	StdDev	Speedup
Python UDF (3 функции)	1.27	1.27	0.017	1.0x (baseline)
Native Functions	0.71	0.70	0.045	1.8x

Анализ

При комплексной обработке native функции дают **ускорение в 1.8 раза**. Разница меньше, чем в bench3, потому что:

- 1. Arrow serialization включен (`spark.sql.execution.arrow.pyspark.enabled = true`)
- 2. Операции более сложные (JSON parsing, string matching)

Сводная таблица результатов

Бенчмарк	Антипаттерн	Оптимизация	Speedup
Join Strategy	Shuffle Join	Broadcast Join	2.2x
Partitioning	200 мелких партиций	Coalesce до 12	6.8x
Partitioning	Repartition (shuffle)	Coalesce (no shuffle)	4.4x
Simple Transform	Python UDF	Native Functions	2.3x
Complex Transform	Python UDF chain	Native Functions	1.8x